Introduction to Programming

Lecture 10

VU
Virtual University

Today's Lecture Includes

- Header Files
- Scope of Variables
- Functions
  - Call by value
  - Call by reference

Header Files

```
#include <iostream.h>
```

Prototype

Return value

Assignment List with data type

```
int functionName ( int , int );
```

Using Header Files

double pi = 3.1415926;

- It is better to define this value in a header file
- Then simply by including the header file in the program this value is defined and it has a meaningful name

#define

- #define pi 3.1415926
- Name can be used inside a program exactly like a variable
- It cannot be used as a variable

CircleArea = pi * radius * radius

Circumference = 2 * pi * radius

VU Virtual University

Scope of Identifiers

- Identifier is any name user creates in his/her program
- Functions are also identifiers
- Labels are also identifiers

Scope of Identifiers

- Scope means visibility
- A variable declared inside a block has visibility within that block only
- Variables defined within the function has a scope that is function wide

Example

```
void functionName ( )
{
  {
   int i ;
  }
 .....
}
```

Identifiers Important Points

- Do not create variables with same name inside blocks, inside functions or inside bigger blocks
- Try to use separate variable names to avoid confusion
- Reuse of variables is valid

File Scope

```
# include < iostream.h >
int i ;
```

Global variable

Global Variable

- Can be used anywhere in program
- Can cause logical problems if same variable name is used in local variable declarations

For good programming

- Try to minimize the use of global variables
- Try to use local variables as far as possible

VU Virtual University

Visibility of Identifiers

- **Global Scope**

  Anything identified or declared outside of any function is visible to all functions in that file

- **Function level scope**

  Declaring variables inside a function can be used in the whole function

- **Block level scope**

  Variables or integers declared inside block are used inside block

Example: Block Scope

```
for ( int i = 0 ; i < 10 ; i++ )
```

- It is block level scope declared in **for** loop
- When **for** is finished " i " no longer exists

Example: Global Scope

```
#include < iostream.h >
int i ;
void f ( void ) ;
main ( )
{
  i = 10 ;
  cout<< " within main i = " << i ;
  f ( ) ;
}
```

Example: Global Scope

```
void f ( void )
{
  cout<< " Inside function f , i =" << i ;
  i = 20 ;
}
```

Example: Call by Value

```
#include <iostream.h >
int f ( int ) ;
main ( )
{
  int i = 10 ;
  cout << “In main i = " << i ;
  f ( i ) ;
  cout << " Back in main,  i = " << i ;
}
```

Example: Call by Value

```
int  f ( int i )
{
  cout << "In function f , i = " << i ;
  i *= 2 ;
  cout << "In function f , i is now = “ << i ;
  return i ;
}
```

Example : Square of a Number

```
double square ( double x )
{
 return x * x ;
}
main ( )
{
 double number = 123.456 ;
 cout << " The square of " << number << " is  "<< square ( number ) ;
 cout << " The current value of  " << number << "is " << number ;
}
```

Math.h

#include < math.h >

double sqrt (double);

$\log_{10}$, pow (x^y) , sin , cos , tan …

Call by Reference

- A function in which original value of the variable is changed
- To call by reference we cannot pass value, we have to pass memory address of variable
- “&” is used to take the address of a variable

Example: Call by Reference

```
main ( )
{
  double x = 123.456 ;
  square ( &x ) ;
}
```

Value of ‘x’ is not passed , but the memory address of ‘x’ is passed

Example: Call by Reference

x is a pointer to a variable double

```
square ( double *x )
{
    *x = *x * *x ;
}
```

Pointers

- Pointers are used to pass address of variable for reference
- We use “ &x ” to send the address of “ x “
- To receive the address we use “ *x ” (whatever “ x ” points to)

Recursive Functions

- Special function which can call itself

$$x^{10} = x * x^{9}$$
$$x^{9} = x * x^{8}$$
$$x^{8} = x * x^{7}$$

… …

$$x^{n} = x * x^{n-1}$$

Recursive Functions: Factorial

n! = n * (n-1) * (n-2) …….. 3 * 2 * 1

5! = 5 * 4 * 3 * 2 * 1

4! = 4 * 3 * 2 * 1

5! = 5 * 4!

0! = 1

Recursive Functions: Factorial

```
long factorial ( long n )
{
  if (n == 1 )
   return ( n ) ;
  else
   return ( n * factorial (n-1) ) ;
}
```

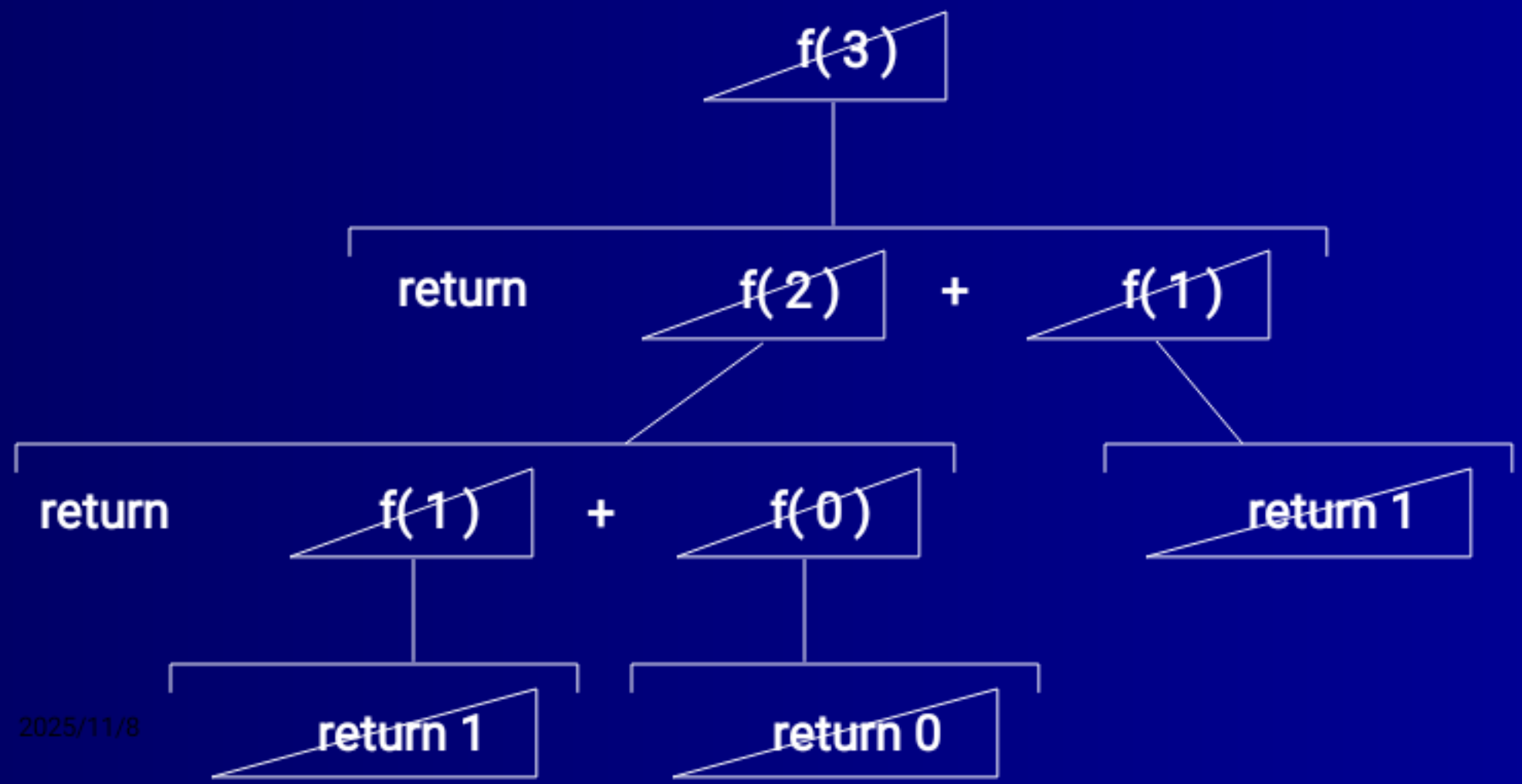
Exercise

Try to write program for

- Fibonacci series
- Find ‘power of number’ using recursive technique

Example
The Fibonacci Series

- Set of recursive calls to function **fibonacci**



Management Issues of Computer

There are two issues inside a computer

- Memory overhead
- Stack overhead

Programming Options

- **Elegant code**
  where price is not too high
- **Efficient code** ✓
  where price is too high

What have we Done Today …

- Header Files
  - Nice mechanism of putting all prototypes and definitions of global constants etc.
- Scope of variables
- Functions
  - Call by value
  - Call by reference
- Recursion